

```
1 from google.colab import drive
2 drive.mount('/content/drive')
```

```
1 import numpy as np
2 import cv2
3 import matplotlib.pyplot as plt
4 import os
5 import glob
```

```

73         (1 - alpha) * beta * image[y1, x0] + \
74         alpha * beta * image[y1, x1]
75     edge_image[y+h, x+w] = interpolated_color.astype(np.uint8)
76     return edge_image, h, w
77
78     def remove_black_background(self, img):
79         mask = img.sum(axis=2) > 0
80         y, x = np.where(mask)
81         x_min, x_max = x.min(), x.max()
82         y_min, y_max = y.min(), y.max()
83         img_cropped = img[y_min:y_max+1, x_min:x_max+1, :]
84         return img_cropped
85
86     def combined_images(self, img1, img2, H, focus=2, blend=True):
87         h1, w1 = img1.shape[:2]
88         h2, w2 = img2.shape[:2]
89         corners1 = np.array([[0, 0], [0, h1], [w1, h1], [w1, 0]], dtype=np.float32)
90         corners2 = np.array([[0, 0], [0, h2], [w2, h2], [w2, 0]], dtype=np.float32)
91
92         corners2_transformed = cv2.perspectiveTransform(corners2.reshape(1, -1, 2), H).reshape(-1, 2)
93         corners = np.concatenate((corners1, corners2_transformed), axis=0)
94         x_min, y_min = np.int32(corners.min(axis=0).ravel() - 0.5)
95         x_max, y_max = np.int32(corners.max(axis=0).ravel() + 0.5)
96
97         T = np.array([[1, 0, -x_min], [0, 1, -y_min], [0, 0, 1]])
98         H_inv = np.linalg.inv(T.dot(H)) # inverse of the homography matrix
99
100        if focus == 1:
101            img_transformed, h_, w_ = self.edges_images(img2, H)
102            img_result = img_transformed.copy()
103            img_result[h_ : h1 + h_, w_ : w1 + w_] = img1
104
105            if blend:
106                img_re_edge = img_result[h_ : h_ + h1, -5 + w_ : w_ + 5]
107                img_result[h_:h+h1, -5 + w_ : w_ + 5] = cv2.GaussianBlur(img_re_edge, (3, 1), 5, 5)
108                img_re_edge = img_result[h_ : h_ + h1, w2 - 5 + w_ : w_ + 5 + w2]
109                img_result[h_ : h_ + h1, w2 - 5 + w_ : w_ + 5 + w2] = cv2.GaussianBlur(img_re_edge, (3, 1), 5, 5)
110                img_re_edge = img_result[-5 + h_ : h_ + 5, w_ : w_ + w1]
111                img_result[-5 + h_ : h_ + 5, w_ : w_ + w1] = cv2.GaussianBlur(img_re_edge, (1, 3), 5, 5)
112                img_re_edge = img_result[h2 - 5 + h_ : h_ + 5 + h2, w_ : w_ + w1]
113                img_result[h2 - 5 + h_ : h_ + 5 + h2, w_ : w_ + w1] = cv2.GaussianBlur(img_re_edge, (1, 3), 5, 5)
114
115            else:
116                img_transformed, h_, w_ = self.edges_images(img1, H_inv)
117                img_result = img_transformed.copy()
118                img_result[-y_min + h_ : h2 - y_min + h_, -x_min + w_ : w2 - x_min + w_] = img2
119
120            if blend:
121                img_re_edge = img_result[-y_min + h_ : h_ + h2 - y_min, -x_min - 5 + w_ : w_ + 5 - x_min]
122                img_result[-y_min + h_:h+h2 - y_min, -x_min -5+ w_:w+5 - x_min] = cv2.GaussianBlur(img_re_edge, (3, 1), 5, 5)
123                img_re_edge = img_result[-y_min + h_ : h_ + h2 - y_min, -x_min + w2 - 5+ w_ : w_ + 5 - x_min + w2]
124                img_result[-y_min + h_ : h_ + h2 - y_min, - x_min + w2 - 5+ w_ : w_ + 5 - x_min + w2] = cv2.GaussianBlur(img_re_edge, (3, 1), 5, 5)
125                img_re_edge = img_result[-y_min - 5 + h_ : h_ + 5 - y_min, -x_min + w_ : w_ + w2 - x_min]
126                img_result[-y_min - 5 + h_ : h_ + 5 - y_min, - x_min + w_ : w_ + w2 - x_min] = cv2.GaussianBlur(img_re_edge, (1, 3), 5, 5)
127                img_re_edge = img_result[-y_min + h2 - 5 + h_ : h_ + 5 - y_min + h2, - x_min + w_ : w_ + w2 - x_min]
128                img_result[-y_min + h2 - 5 + h_ : h_ + 5 - y_min + h2, -x_min + w_ : w_ + w2 - x_min] = cv2.GaussianBlur(img_re_edge, (1, 3), 5, 5)
129
130        return img_result

```

```

1 def show_image(img, w=0, h=0, axis=False, title=''):
2     if w == 0 or h == 0: plt.figure()
3     else: plt.figure(figsize=(w, h)) # set the size of the figure
4     plt.title(title) # set the title of the figure
5     plt.imshow(img) # show the image
6     if axis == False: plt.axis('off') # turn off the axis
7     plt.show()
8
9 def open_resize(img, w, h):
10    image = cv2.imread(img) # read the image
11    image = cv2.resize(image, (w, h)) # resize the image
12    return image

```

```
1 fun = PanoramaStitcher()
```

✓ path => I1

```

1 path1 = '/content/drive/MyDrive/Colab Notebooks/content/Images/I1/STA_0031.JPG'
2 path2 = '/content/drive/MyDrive/Colab Notebooks/content/Images/I1/STB_0032.JPG'
3 path3 = '/content/drive/MyDrive/Colab Notebooks/content/Images/I1/STC_0033.JPG'
4 path4 = '/content/drive/MyDrive/Colab Notebooks/content/Images/I1/STD_0034.JPG'
5 path5 = '/content/drive/MyDrive/Colab Notebooks/content/Images/I1/STE_0035.JPG'
6 path6 = '/content/drive/MyDrive/Colab Notebooks/content/Images/I1/STF_0036.JPG'

```

```

7 image1 = open_resize(path1, 600 , 400)
8 image2 = open_resize(path2, 600 , 400)
9 image3 = open_resize(path3, 600 , 400)
10 image4 = open_resize(path4, 600 , 400)
11 image5 = open_resize(path5, 600 , 400)
12 image6 = open_resize(path6, 600 , 400)
13 homography_matrix_list = []

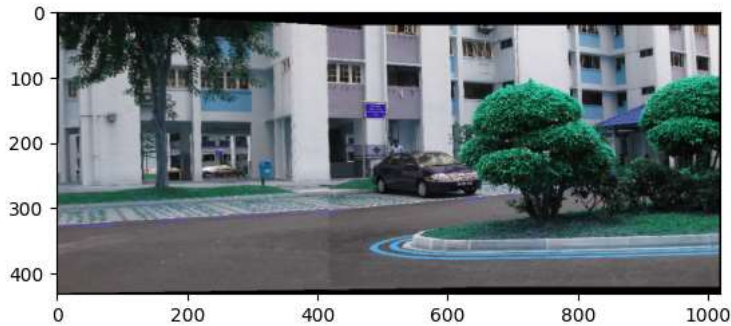
1 source_points_1, destination_points_1, matching_result_1 = fun.source_destination_points(image1, image2)
2 _H1 = fun.ransac(source_points_1, destination_points_1)
3 merged_image_1 = fun.combined_images(image1, image2, _H1, focus=2, blend=True)
4 merged_image_1 = fun.remove_black_background((merged_image_1))
5 homography_matrix_list.append(_H1)
6 print(_H1)
7 show_image(merged_image_1, axis=True)

```

```

[[ [ 1.11874852e+00 -3.77567173e-02 -4.13950532e+02]
   [ 6.074444563e-02 1.10939829e+00 -2.28595627e+01]
   [ 1.89478252e-04 6.41825935e-05 1.00000000e+00]]

```



```

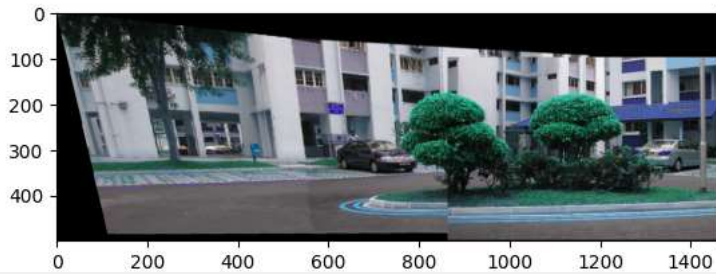
1 source_points_2, destination_points_2, matching_result_2 = fun.source_destination_points(merged_image_1, image3)
2 _H2 = fun.ransac(source_points_2, destination_points_2)
3 merged_image_2 = fun.combined_images(merged_image_1, image3, _H2, focus=2, blend=True)
4 merged_image_2 = fun.remove_black_background((merged_image_2))
5 homography_matrix_list.append(_H2)
6 print(_H2)
7 show_image(merged_image_2, axis=True)

```

```

[[ [ 1.20853733e+00 2.03782829e-03 -8.70101817e+02]
   [ 6.66095333e-02 1.26949604e+00 -9.96585347e+01]
   [ 1.43339288e-04 3.63773319e-04 1.00000000e+00]]

```



```

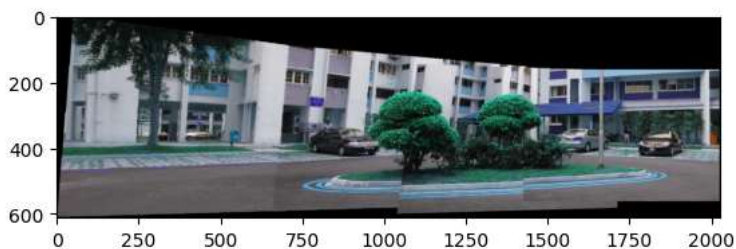
1 source_points_3, destination_points_3, matching_result_3 = fun.source_destination_points(merged_image_2, image4)
2 _H3 = fun.ransac(source_points_3, destination_points_3)
3 merged_image_3 = fun.combined_images(merged_image_2, image4, _H3, focus=2, blend=True)
4 merged_image_3 = fun.remove_black_background((merged_image_3))
5 homography_matrix_list.append(_H3)
6 print(_H3)
7 show_image(merged_image_3, axis=True)

```

```

[[ [ 1.18398974e+00 -8.12688192e-02 -1.37346080e+03]
   [ 3.61858082e-02 1.1557798e+00 -1.63645647e+02]
   [ 1.58269218e-04 -2.46924278e-04 1.00000000e+00]]

```



```

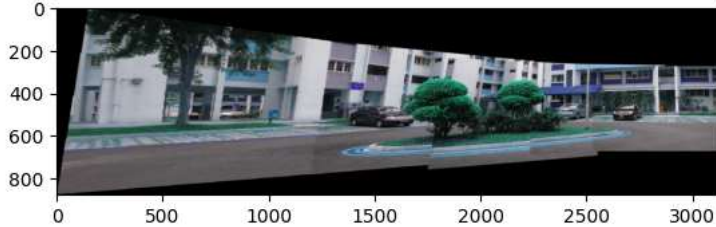
1 source_points_4, destination_points_4, matching_result_4 = fun.source_destination_points(merged_image_3, image5)
2 _H4 = fun.ransac(source_points_4, destination_points_4)
3 merged_image_4 = fun.combined_images(merged_image_3, image5, _H4, focus=2, blend=True)
4 merged_image_4 = fun.remove_black_background((merged_image_4))
5 homography_matrix_list.append(_H4)
6 print(_H4)
7 show_image(merged_image_4, axis=True)

```

```

[[ [ 1.47971405e+00 -2.95702136e-02 -2.48381608e+03]
   [ 1.84333281e-02 1.43280526e+00 -2.78022036e+02]
   [ 2.38573690e-04 -1.80562378e-05 1.00000000e+00]]

```



```

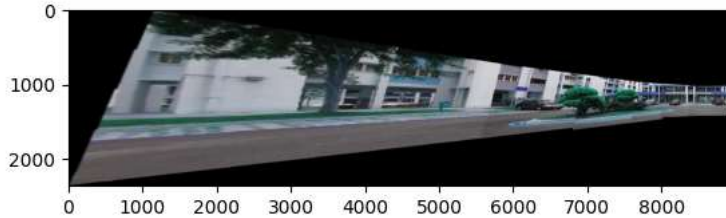
1 source_points_5, destination_points_5, matching_result_5 = fun.source_destination_points(merged_image_4, image6)
2 _H5 = fun.ransac(source_points_5, destination_points_5)
3 merged_image_5 = fun.combined_images(merged_image_4, image6, _H5, focus=2, blend=True)
4 merged_image_5 = fun.remove_black_background((merged_image_5))
5 homography_matrix_list.append(_H5)
6 print(_H5)
7 show_image(merged_image_5, axis=True)

```

```

[[ [ 2.90696117e+00 -4.82907341e-02 -8.28382396e+03]
   [ 1.22897340e-01 2.81961432e+00 -1.13548246e+03]
   [ 6.09563716e-04 -3.20256933e-06 1.00000000e+00]]

```



Final I1

```

1 print((homography_matrix_list))
2 show_image(merged_image_5, axis=False)

```

```

[[array([[ 1.11874852e+00, -3.77567173e-02, -4.13950532e+02],
         [ 6.07444563e-02, 1.10939829e+00, -2.28595627e+01],
         [ 1.89478252e-04, 6.41825935e-05, 1.00000000e+00]]), array([[ 1.20853733e+00, 2.03782829e-03, -8.70101817e+02],
         [ 6.66095333e-02, 1.26949604e+00, -9.96585347e+01],
         [ 1.43339288e-04, 3.63773319e-04, 1.00000000e+00]]), array([[ 1.18398974e+00, -8.12688192e-02, -1.37346080e+03],
         [ 3.61858082e-02, 1.1557798e+00, -1.63645647e+02],
         [ 1.58269218e-04, -2.46924278e-04, 1.00000000e+00]]), array([[ 1.47971405e+00, -2.95702136e-02, -2.48381608e+03],
         [ 1.84333281e-02, 1.43280526e+00, -2.78022036e+02],
         [ 2.38573690e-04, -1.80562378e-05, 1.00000000e+00]]), array([[ 2.90696117e+00, -4.82907341e-02, -8.28382396e+03],
         [ 1.22897340e-01, 2.81961432e+00, -1.13548246e+03],
         [ 6.09563716e-04, -3.20256933e-06, 1.00000000e+00]])]]

```



path => I2

```

1 path1 = '/content/drive/MyDrive/Colab Notebooks/content/Images/I2/2_1.JPG'
2 path2 = '/content/drive/MyDrive/Colab Notebooks/content/Images/I2/2_2.JPG'
3 path3 = '/content/drive/MyDrive/Colab Notebooks/content/Images/I2/2_3.JPG'

```

```

4 path4 = '/content/drive/MyDrive/Colab Notebooks/content/Images/I2/2_4.JPG'
5 path5 = '/content/drive/MyDrive/Colab Notebooks/content/Images/I2/2_5.JPG'
6 image1 = open_resize(path1, 600 , 400)
7 image2 = open_resize(path2, 600 , 400)
8 image3 = open_resize(path3, 600 , 400)
9 image4 = open_resize(path4, 600 , 400)
10 image5 = open_resize(path5, 600 , 400)
11 homography_matrix_list = []

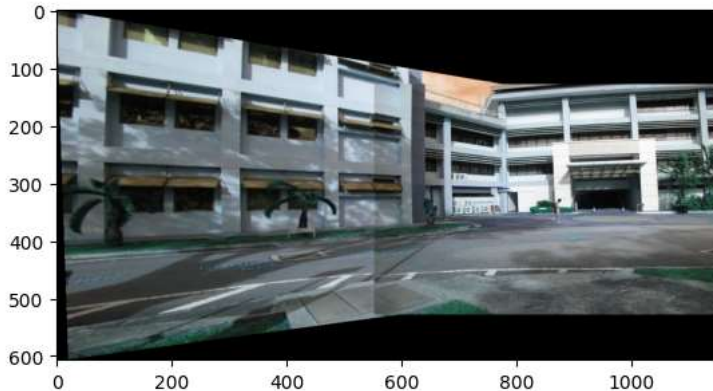
1 source_points_1, destination_points_1, matching_result_1 = fun.source_destination_points(image1, image2)
2 _H1 = fun.ransac(source_points_1, destination_points_1)
3 mergered_image_1 = fun.combined_images(image1, image2, _H1, focus=2, blend=True)
4 mergered_image_1 = fun.remove_black_background((mergered_image_1))
5 homography_matrix_list.append(_H1)
6 print(_H1)
7 show_image(mergered_image_1, axis=True)

```

```

[[ [ 1.73840967e+00 -1.65960173e-02 -5.53802592e+02]
  [ 2.44693084e-01 1.58360466e+00 -1.29913005e+02]
  [ 1.18249342e-03 1.28673898e-04 1.00000000e+00]]

```



```

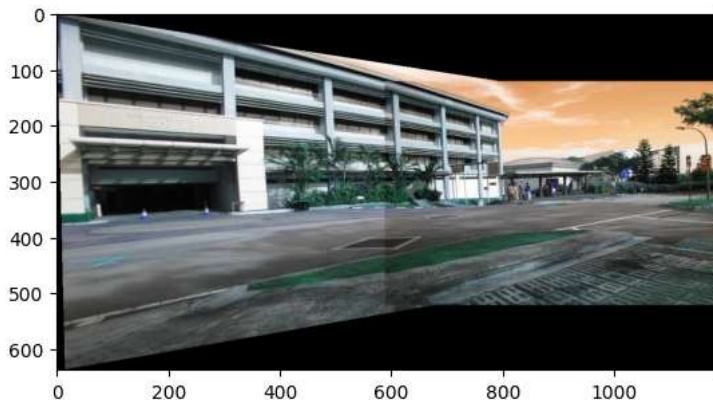
1 source_points_2, destination_points_2, matching_result_2 = fun.source_destination_points(image3, image4)
2 _H2 = fun.ransac(source_points_2, destination_points_2)
3 mergered_image_2 = fun.combined_images(image3, image4, _H2, focus=2, blend=True)
4 mergered_image_2 = fun.remove_black_background((mergered_image_2))
5 homography_matrix_list.append(_H2)
6 print(_H2)
7 show_image(mergered_image_2, axis=True)

```

```

[[ [ 1.80824811e+00 3.17877117e-03 -5.89535676e+02]
  [ 2.34675144e-01 1.62795570e+00 -1.22399385e+02]
  [ 1.32416599e-03 6.00934644e-05 1.00000000e+00]]

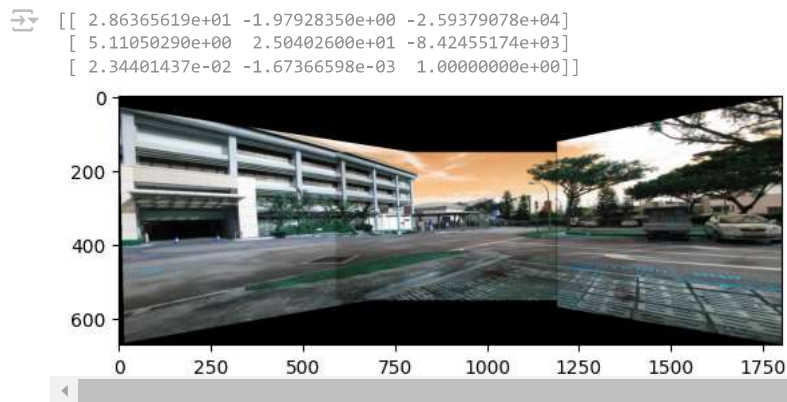
```



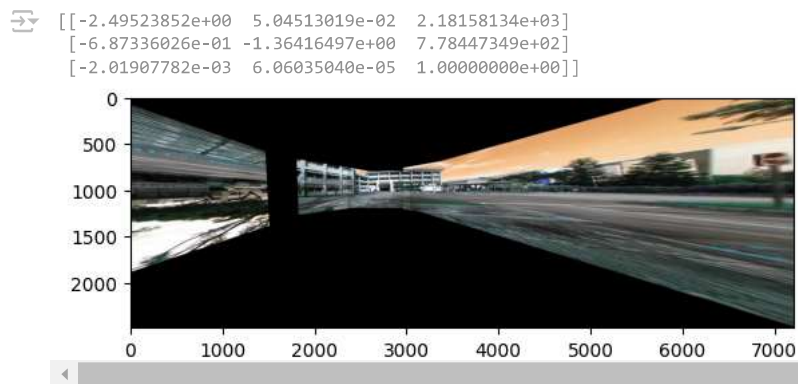
```

1 source_points_3, destination_points_3, matching_result_3 = fun.source_destination_points(mergered_image_2, image5)
2 _H3 = fun.ransac(source_points_3, destination_points_3)
3 mergered_image_3 = fun.combined_images(mergered_image_2, image5, _H3, focus=1 , blend=True)
4 mergered_image_3 = fun.remove_black_background((mergered_image_3))
5 homography_matrix_list.append(_H3)
6 print(_H3)
7 show_image(mergered_image_3, axis=True)

```



```
1 source_points_4, destination_points_4, matching_result_4 = fun.source_destination_points(merged_image_1, merged_image_3)
2 _H4 = fun.ransac(source_points_4, destination_points_4)
3 merged_image_4 = fun.combined_images(merged_image_1, merged_image_3, _H4, focus=1, blend=False)
4 merged_image_4 = fun.remove_black_background((merged_image_4))
5 homography_matrix_list.append(_H4)
6 print(_H4)
7 show_image(merged_image_4, axis=True)
```



Final I2

```
1 print((homography_matrix_list))
2 show_image(merged_image_4, axis=False)
```

```
[array([[ 1.73840967e+00, -1.65960173e-02, -5.53802592e+02],
        [ 2.44693084e-01,  1.58360466e+00, -1.29913005e+02],
        [ 1.18249342e-03,  1.28673898e-04,  1.00000000e+00]]), array([[ 1.80824811e+00,  3.17877117e-03, -5.89535676e+02],
        [ 2.34675144e-01,  1.62795570e+00, -1.22399385e+02],
        [ 1.32416599e-03,  6.00934644e-05,  1.00000000e+00]]), array([[ 2.86365619e+01, -1.97928350e+00, -2.59379078e+04],
        [ 5.11050290e+00,  2.50402600e+01, -8.42455174e+03],
        [ 2.34401437e-02, -1.67366598e-03,  1.00000000e+00]]), array([[ -2.49523852e+00,  5.04513019e-02,  2.18158134e+03],
        [-6.87336026e-01, -1.36416497e+00,  7.78447349e+02],
        [-2.01907782e-03,  6.06035040e-05,  1.00000000e+00]])]
```



path => I3

```
1 path1 = '/content/drive/MyDrive/Colab Notebooks/content/Images/I3/3_1.JPG'
2 path2 = '/content/drive/MyDrive/Colab Notebooks/content/Images/I3/3_2.JPG'
3 path3 = '/content/drive/MyDrive/Colab Notebooks/content/Images/I3/3_3.JPG'
4 path4 = '/content/drive/MyDrive/Colab Notebooks/content/Images/I3/3_4.JPG'
5 path5 = '/content/drive/MyDrive/Colab Notebooks/content/Images/I3/3_5.JPG'
6 image1 = open_resize(path1, 600 , 400)
7 image2 = open_resize(path2, 600 , 400)
```

```

8 image3 = open_resize(path3, 600 , 400)
9 image4 = open_resize(path4, 600 , 400)
10 image5 = open_resize(path5, 600 , 400)
11 homography_matrix_list = []

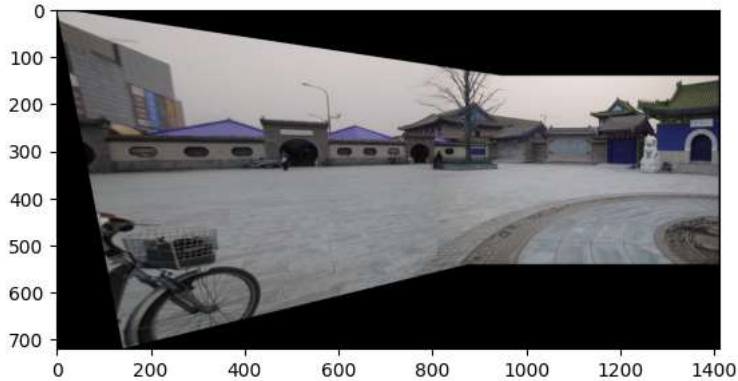
1 source_points_1, destination_points_1, matching_result_1 = fun.source_destination_points(image1, image2)
2 _H1 = fun.ransac(source_points_1, destination_points_1)
3 merged_image_1 = fun.combined_images(image1, image2, _H1, focus=2, blend=True)
4 merged_image_1 = fun.remove_black_background((merged_image_1))
5 homography_matrix_list.append(_H1)
6 print(_H1)
7 show_image(merged_image_1, axis=True)

```

```

[[ [ 2.21554357e+00  7.29456081e-02 -8.11237098e+02]
   [ 2.89861397e-01  2.05018567e+00 -1.42794335e+02]
   [ 1.99740740e-03  4.20153205e-04  1.00000000e+00]]

```



```

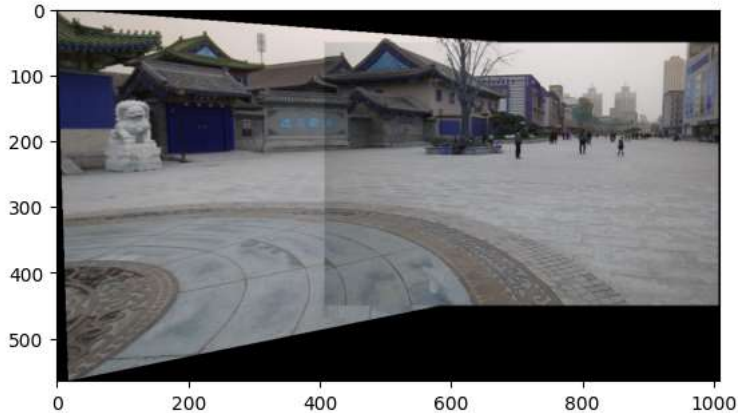
1 source_points_2, destination_points_2, matching_result_2 = fun.source_destination_points(image4, image5)
2 _H2 = fun.ransac(source_points_2, destination_points_2)
3 merged_image_2 = fun.combined_images(image4, image5, _H2, focus=2, blend=True)
4 merged_image_2 = fun.remove_black_background((merged_image_2))
5 homography_matrix_list.append(_H2)
6 print(_H2)
7 show_image(merged_image_2, axis=True)

```

```

[[ [ 1.56374433e+00  6.16954608e-02 -4.08897106e+02]
   [ 9.80480779e-02  1.39670806e+00 -5.15495783e+01]
   [ 9.51498282e-04 -4.00872534e-05  1.00000000e+00]]

```



```

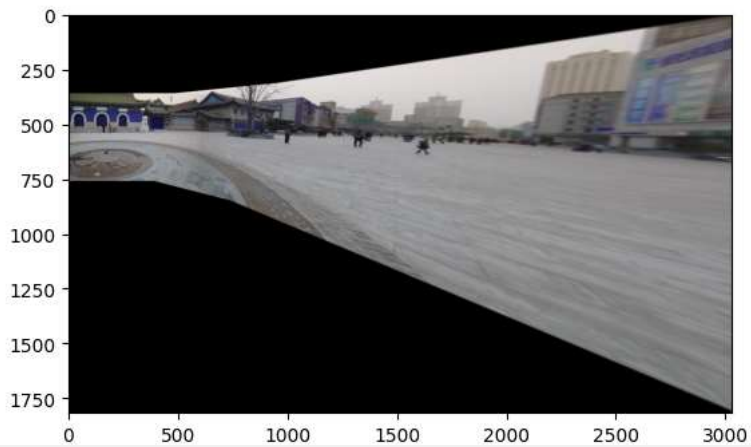
1 source_points_3, destination_points_3, matching_result_3 = fun.source_destination_points(image3, merged_image_2)
2 _H3 = fun.ransac(source_points_3, destination_points_3)
3 merged_image_3 = fun.combined_images(image3, merged_image_2, _H3, focus=1 , blend=True)
4 merged_image_3 = fun.remove_black_background((merged_image_3))
5 homography_matrix_list.append(_H3)
6 print(_H3)
7 show_image(merged_image_3, axis=True)

```

```

→ [[ 7.98201688e+00  5.42609148e-01 -2.43200722e+03]
   [ 1.09869400e+00  5.04046627e+00 -4.22589273e+02]
   [ 6.95603984e-03  4.09677159e-04  1.00000000e+00]]

```



```

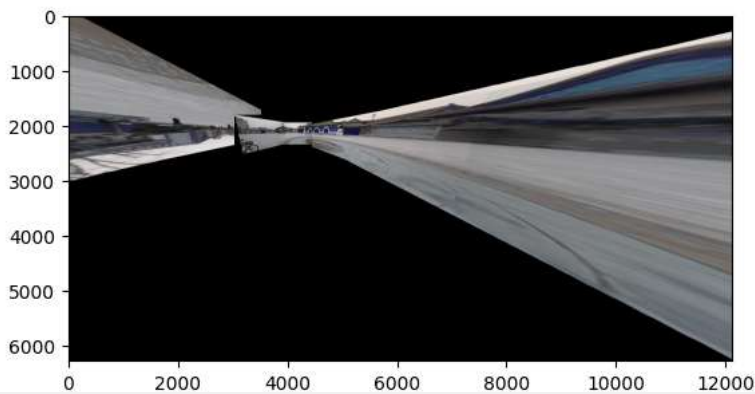
1 source_points_4, destination_points_4, matching_result_4 = fun.source_destination_points(merged_image_1, merged_image_3)
2 _H4 = fun.ransac(source_points_4, destination_points_4)
3 merged_image_4 = fun.combined_images(merged_image_1, merged_image_3, _H4, focus=1, blend=False)
4 merged_image_4 = fun.remove_black_background((merged_image_4))
5 homography_matrix_list.append(_H4)
6 print(_H4)
7 show_image(merged_image_4, axis=True)

```

```

→ [[-1.05490930e+00 -6.14911263e-02  1.41829611e+03]
   [-7.19013967e-01 -1.01449164e+00  8.02507003e+02]
   [-1.45337281e-03 -6.62708315e-06  1.00000000e+00]]

```



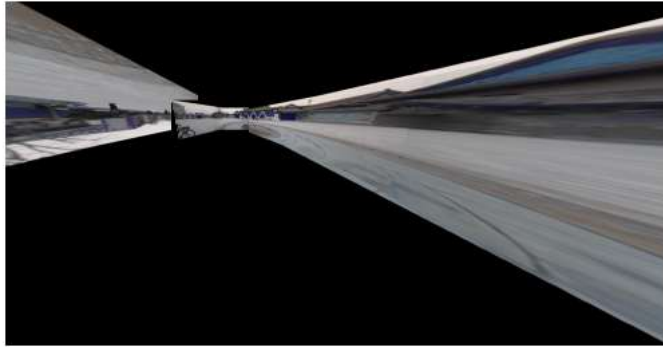
Final I3

```

1 print((homography_matrix_list))
2 show_image(merged_image_4, axis=False)

```

```
[array([[ 2.21554357e+00,  7.29456081e-02, -8.11237098e+02],
        [ 2.89861397e-01,  2.05018567e+00, -1.42794335e+02],
        [ 1.99740740e-03,  4.20153205e-04,  1.00000000e+00]]), array([[ 1.56374433e+00,  6.16954608e-02, -4.08897106e+02],
        [ 9.80480779e-02,  1.39670806e+00, -5.15495783e+01],
        [ 9.51498282e-04, -4.00872534e-05,  1.00000000e+00]]), array([[ 7.98201688e+00,  5.42609148e-01, -2.43200722e+03],
        [ 1.09869400e+00,  5.04046627e+00, -4.22589273e+02],
        [ 6.95603984e-03,  4.09677159e-04,  1.00000000e+00]]), array([[ -1.05490930e+00, -6.14911263e-02,  1.41829611e+03],
        [-7.19013967e-01, -1.01449164e+00,  8.02507003e+02],
        [-1.45337281e-03, -6.62708315e-06,  1.00000000e+00]])]
```

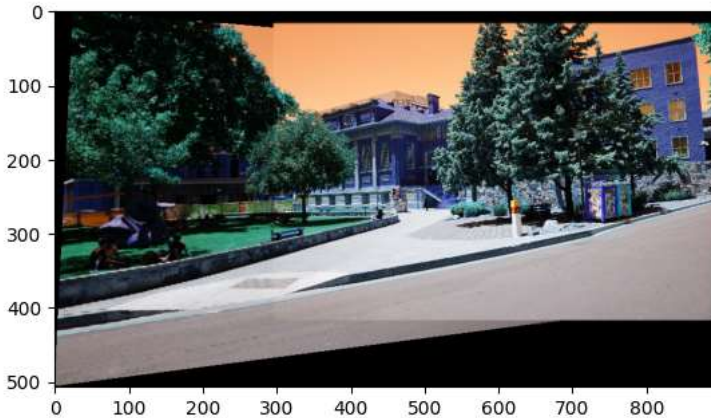


▼ path => I4

```
1 path1 = '/content/drive/MyDrive/Colab Notebooks/content/Images/I4/DSC02930.JPG'
2 path2 = '/content/drive/MyDrive/Colab Notebooks/content/Images/I4/DSC02931.JPG'
3 path3 = '/content/drive/MyDrive/Colab Notebooks/content/Images/I4/DSC02932.JPG'
4 path4 = '/content/drive/MyDrive/Colab Notebooks/content/Images/I4/DSC02933.JPG'
5 path5 = '/content/drive/MyDrive/Colab Notebooks/content/Images/I4/DSC02934.JPG'
6 image1 = open_resize(path1, 600 , 400)
7 image2 = open_resize(path2, 600 , 400)
8 image3 = open_resize(path3, 600 , 400)
9 image4 = open_resize(path4, 600 , 400)
10 image5 = open_resize(path5, 600 , 400)
11 homography_matrix_list = []

1 source_points_1, destination_points_1, matching_result_1 = fun.source_destination_points(image1, image2)
2 _H1 = fun.ransac(source_points_1, destination_points_1)
3 merged_image_1 = fun.combined_images(image1, image2, _H1, focus=2, blend=True)
4 merged_image_1 = fun.remove_black_background((merged_image_1))
5 homography_matrix_list.append(_H1)
6 print(_H1)
7 show_image(merged_image_1, axis=True)
```

```
[ [ 1.36244491e+00 -2.53795651e-02 -2.71632071e+02]
  [ 1.25013351e-01  1.20864680e+00 -1.86746271e+01]
  [ 6.68931631e-04 -1.26135527e-04  1.00000000e+00]]
```



```
1 source_points_2, destination_points_2, matching_result_2 = fun.source_destination_points(merged_image_1, image3)
2 _H2 = fun.ransac(source_points_2, destination_points_2)
3 merged_image_2 = fun.combined_images(merged_image_1, image3, _H2, focus=1, blend=True)
4 merged_image_2 = fun.remove_black_background((merged_image_2))
5 homography_matrix_list.append(_H2)
6 print(_H2)
7 show_image(merged_image_2, axis=True)
```

```

[[ 1.66829515e+00 -4.88352494e-02 -8.15275073e+02]
 [ 1.56669637e-01  1.45972573e+00 -1.05619095e+02]
 [ 7.32189747e-04 -8.19162071e-05  1.00000000e+00]]

```



```

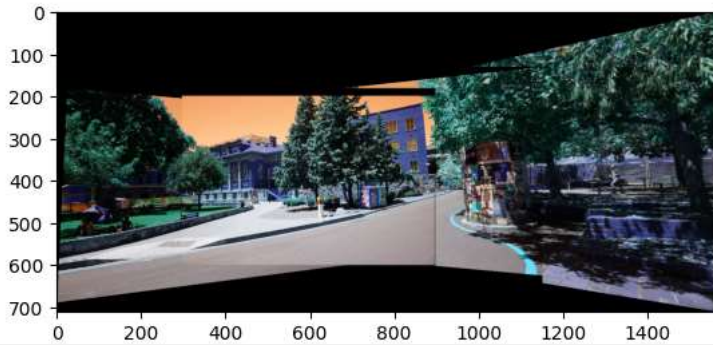
1 source_points_3, destination_points_3, matching_result_3 = fun.source_destination_points(merged_image_2, image4)
2 _H3 = fun.ransac(source_points_3, destination_points_3)
3 merged_image_3 = fun.combined_images(merged_image_2, image4, _H3, focus=1, blend=False)
4 merged_image_3 = fun.remove_black_background((merged_image_3))
5 homography_matrix_list.append(_H3)
6 print(_H3)
7 show_image(merged_image_3, axis=True)

```

```

[[ 3.93708316e+00 -2.61032211e-01 -2.48042189e+03]
 [ 7.43024543e-01  3.30995391e+00 -7.29497253e+02]
 [ 3.29843384e-03 -4.72323790e-04  1.00000000e+00]]

```



```

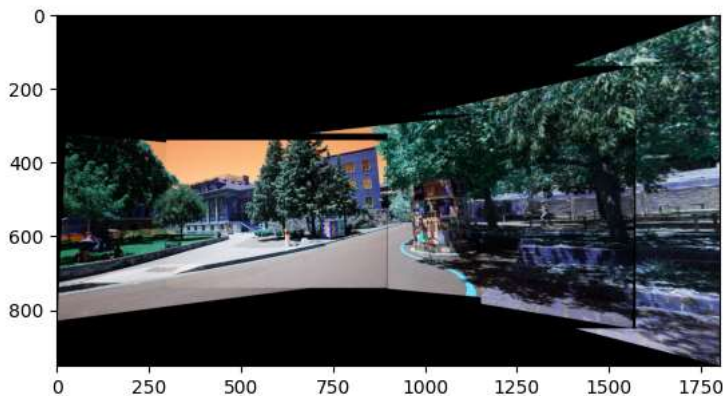
1 source_points_4, destination_points_4, matching_result_4 = fun.source_destination_points(merged_image_3, image5)
2 _H4 = fun.ransac(source_points_4, destination_points_4)
3 merged_image_4 = fun.combined_images(merged_image_3, image5, _H4, focus=1, blend=True)
4 merged_image_4 = fun.remove_black_background((merged_image_4))
5 homography_matrix_list.append(_H4)
6 print(_H4)
7 show_image(merged_image_4, axis=True)

```

```

[[ -3.03268552e+00  6.75538512e-02  2.61576924e+03]
 [ -9.89858520e-01 -2.78857092e+00  1.38702035e+03]
 [ -4.30922879e-03  1.21317895e-04  1.00000000e+00]]

```



```
1 print((homography_matrix_list))
2 show_image(merged_image_4, axis=False)
```

```
[array([[ 1.36244491e+00, -2.53795651e-02, -2.71632071e+02],
        [ 1.25013351e-01,  1.20864680e+00, -1.86746271e+01],
        [ 6.68931631e-04, -1.26135527e-04,  1.00000000e+00]]), array([[ 1.66829515e+00, -4.88352494e-02, -8.15275073e+02],
        [ 1.56669637e-01,  1.45972573e+00, -1.05619095e+02],
        [ 7.32189747e-04, -8.19162071e-05,  1.00000000e+00]]), array([[ 3.93708316e+00, -2.61032211e-01, -2.48042189e+03],
        [ 7.43024543e-01,  3.30995391e+00, -7.29497253e+02],
        [ 3.29843384e-03, -4.72323790e-04,  1.00000000e+00]]), array([[ -3.03268552e+00,  6.75538512e-02,  2.61576924e+03],
        [ -9.89858520e-01, -2.78857092e+00,  1.38702035e+03],
        [ -4.30922879e-03,  1.21317895e-04,  1.00000000e+00]])]
```

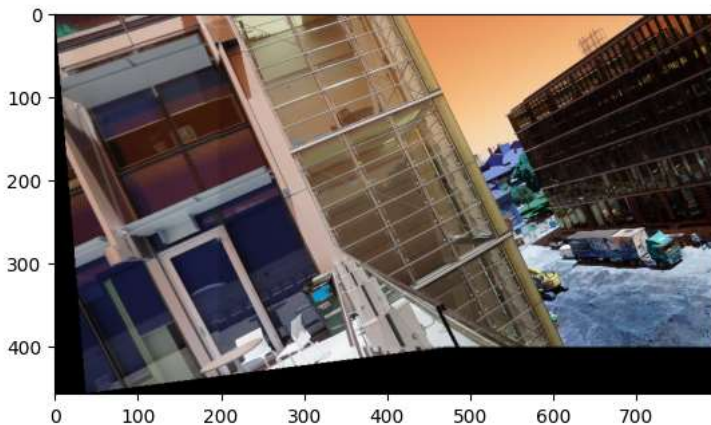


↘ path => I5

```
1 path1 = '/content/drive/MyDrive/Colab Notebooks/content/Images/I5/DSC03002.JPG'
2 path2 = '/content/drive/MyDrive/Colab Notebooks/content/Images/I5/DSC03003.JPG'
3 path3 = '/content/drive/MyDrive/Colab Notebooks/content/Images/I5/DSC03004.JPG'
4 path4 = '/content/drive/MyDrive/Colab Notebooks/content/Images/I5/DSC03005.JPG'
5 path5 = '/content/drive/MyDrive/Colab Notebooks/content/Images/I5/DSC03006.JPG'
6 image1 = open_resize(path1, 600 , 400)
7 image2 = open_resize(path2, 600 , 400)
8 image3 = open_resize(path3, 600 , 400)
9 image4 = open_resize(path4, 600 , 400)
10 image5 = open_resize(path5, 600 , 400)
11 homography_matrix_list = []

1 source_points_1, destination_points_1, matching_result_1 = fun.source_destination_points(image1, image2)
2 _H1 = fun.ransac(source_points_1, destination_points_1)
3 merged_image_1 = fun.combined_images(image1, image2, _H1, focus=2, blend=True)
4 merged_image_1 = fun.remove_black_background((merged_image_1))
5 homography_matrix_list.append(_H1)
6 print(_H1)
7 show_image(merged_image_1, axis=True)
```

```
[ [ 1.28380178e+00  8.43622294e-02 -1.98199899e+02]
  [ 2.19633387e-02  1.17350177e+00 -1.97264129e+00]
  [ 4.28003857e-04  6.13009149e-05  1.00000000e+00]]
```



```
1 source_points_2, destination_points_2, matching_result_2 = fun.source_destination_points(merged_image_1, image3)
2 _H2 = fun.ransac(source_points_2, destination_points_2)
3 merged_image_2 = fun.combined_images(merged_image_1, image3, _H2, focus=2, blend=True)
4 merged_image_2 = fun.remove_black_background((merged_image_2))
```

```

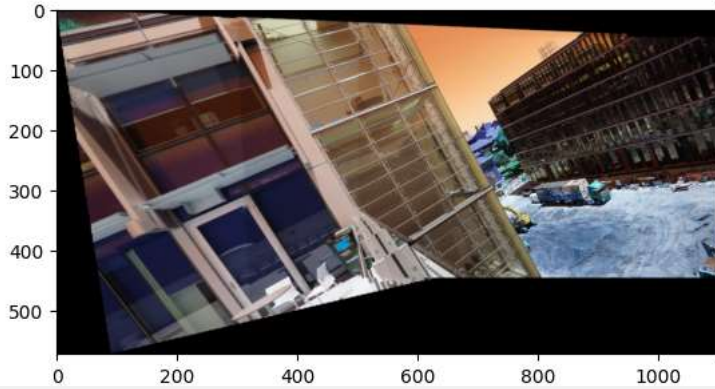
5 homography_matrix_list.append(_H2)
6 print(_H2)
7 show_image(merged_image_2, axis=True)

```

```

[[ [ 1.41840075e+00  5.09574490e-02 -5.02973328e+02]
   [ 4.97143659e-02  1.29481125e+00 -4.79024527e+01]
   [ 5.05885011e-04  3.78627861e-05  1.00000000e+00]]

```



```

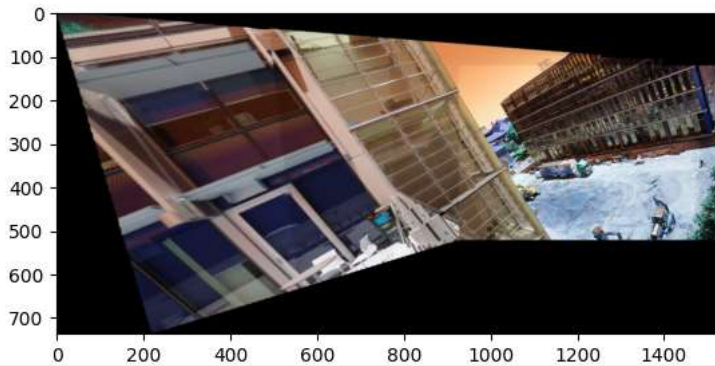
1 source_points_3, destination_points_3, matching_result_3 = fun.source_destination_points(merged_image_2, image4)
2 _H3 = fun.ransac(source_points_3, destination_points_3)
3 merged_image_3 = fun.combined_images(merged_image_2, image4, _H3, focus=2, blend=True)
4 merged_image_3 = fun.remove_black_background((merged_image_3))
5 homography_matrix_list.append(_H3)
6 print(_H3)
7 show_image(merged_image_3, axis=True)

```

```

[[ [ 1.45368806e+00  5.39148182e-02 -9.28076156e+02]
   [ 3.47333044e-02  1.36981867e+00 -1.23704959e+02]
   [ 3.98553627e-04  7.85227666e-05  1.00000000e+00]]

```



```

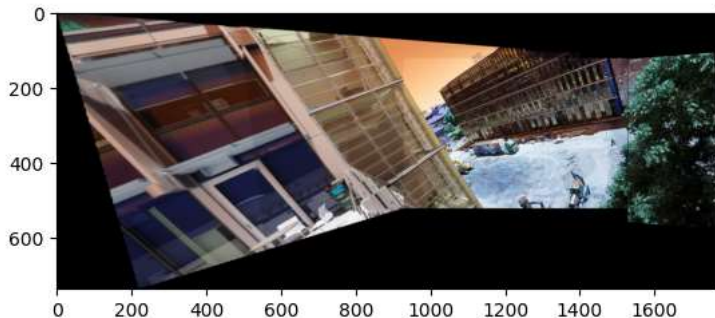
1 source_points_4, destination_points_4, matching_result_4 = fun.source_destination_points(merged_image_3, image5)
2 _H4 = fun.ransac(source_points_4, destination_points_4)
3 merged_image_4 = fun.combined_images(merged_image_3, image5, _H4, focus=1, blend=True)
4 merged_image_4 = fun.remove_black_background((merged_image_4))
5 homography_matrix_list.append(_H4)
6 print(_H4)
7 show_image(merged_image_4, axis=True)

```

```

[[ [ 2.34451952e+00 -2.82664736e-03 -2.60410959e+03]
   [ 1.64324776e-01  2.18370037e+00 -5.23506755e+02]
   [ 8.70057521e-04  8.29665921e-05  1.00000000e+00]]

```



✓ Final I5

```
1 print((homography_matrix_list))
2 show_image(merged_image_4, axis=False)
```

```
[array([[ 1.28380178e+00,  8.43622294e-02, -1.98199899e+02],
        [ 2.19633387e-02,  1.17350177e+00, -1.97264129e+00],
        [ 4.28003857e-04,  6.13009149e-05,  1.00000000e+00]]), array([[ 1.41840075e+00,  5.09574490e-02, -5.02973328e+02],
        [ 4.97143659e-02,  1.29481125e+00, -4.79024527e+01],
        [ 5.05885011e-04,  3.78627861e-05,  1.00000000e+00]]), array([[ 1.45368806e+00,  5.39148182e-02, -9.28076156e+02],
        [ 3.47333044e-02,  1.36981867e+00, -1.23704959e+02],
        [ 3.98553627e-04,  7.85227666e-05,  1.00000000e+00]]), array([[ 2.34451952e+00, -2.82664736e-03, -2.60410959e+03],
        [ 1.64324776e-01,  2.18370037e+00, -5.23506755e+02],
        [ 8.70057521e-04,  8.29665921e-05,  1.00000000e+00]])]
```

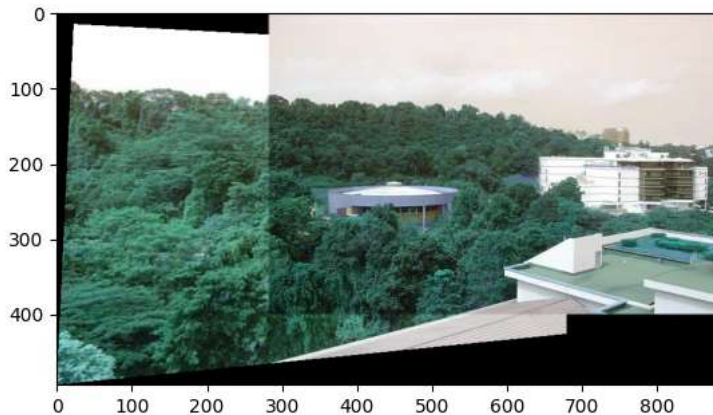


✓ path => I6

```
1 path1 = '/content/drive/MyDrive/Colab Notebooks/content/Images/I6/1_1.JPG'
2 path2 = '/content/drive/MyDrive/Colab Notebooks/content/Images/I6/1_2.JPG'
3 path3 = '/content/drive/MyDrive/Colab Notebooks/content/Images/I6/1_3.JPG'
4 path4 = '/content/drive/MyDrive/Colab Notebooks/content/Images/I6/1_4.JPG'
5 path5 = '/content/drive/MyDrive/Colab Notebooks/content/Images/I6/1_5.JPG'
6 image1 = open_resize(path1, 600 , 400)
7 image2 = open_resize(path2, 600 , 400)
8 image3 = open_resize(path3, 600 , 400)
9 image4 = open_resize(path4, 600 , 400)
10 image5 = open_resize(path5, 600 , 400)
11 homography_matrix_list = []

1 source_points_1, destination_points_1, matching_result_1 = fun.source_destination_points(image1, image2)
2 _H1 = fun.ransac(source_points_1, destination_points_1)
3 merged_image_1 = fun.combined_images(image1, image2, _H1, focus=2, blend=True)
4 merged_image_1 = fun.remove_black_background((merged_image_1))
5 homography_matrix_list.append(_H1)
6 print(_H1)
7 show_image(merged_image_1, axis=True)
```

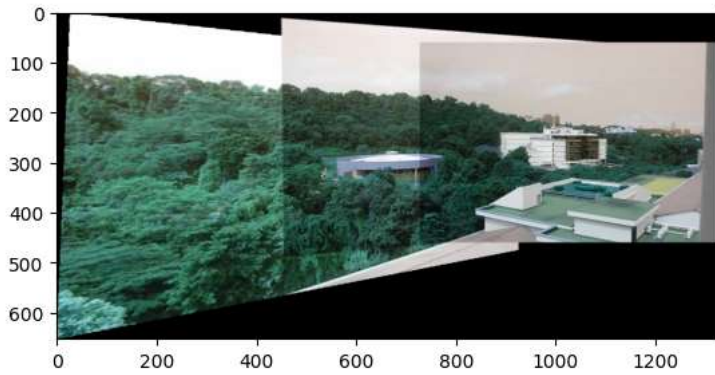
```
[ [ 1.25680531e+00 -2.33362627e-02 -2.60638267e+02]
  [ 8.03411856e-02  1.14041527e+00  1.47499222e+01]
  [ 4.41173699e-04 -1.17344720e-04  1.00000000e+00]]
```



```
1 source_points_2, destination_points_2, matching_result_2 = fun.source_destination_points(merged_image_1, image3)
2 _H2 = fun.ransac(source_points_2, destination_points_2)
3 merged_image_2 = fun.combined_images(merged_image_1, image3, _H2, focus=2, blend=True)
4 merged_image_2 = fun.remove_black_background((merged_image_2))
5 homography_matrix_list.append(_H2)
```

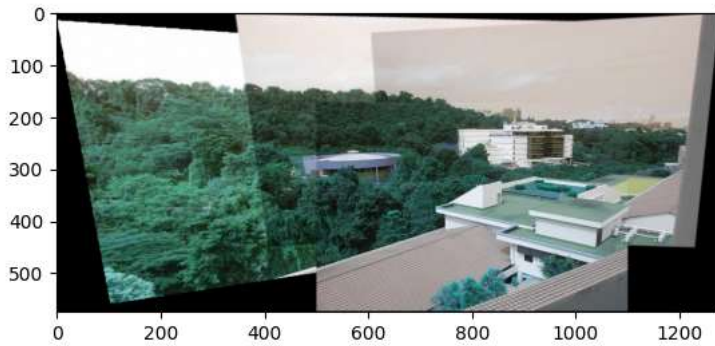
```
6 print(_H2)
7 show_image(merged_image_2, axis=True)
```

```
→ [[ 1.49735477e+00  2.77822180e-03 -7.44066307e+02]
 [ 9.72082085e-02  1.39715295e+00 -8.50357001e+01]
 [ 5.49360049e-04  4.12790562e-05  1.00000000e+00]]
```



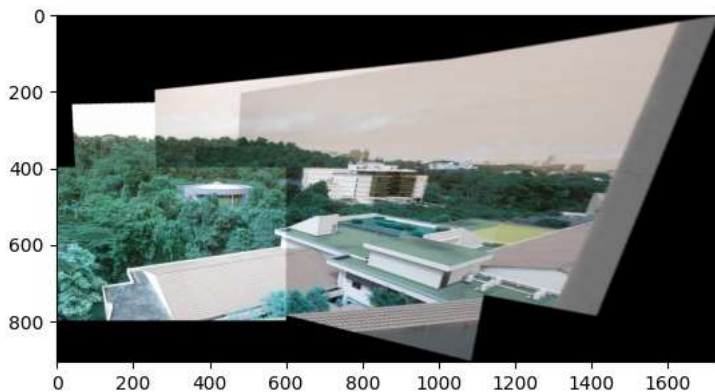
```
1 source_points_3, destination_points_3, matching_result_3 = fun.source_destination_points(merged_image_2, image4)
2 _H3 = fun.ransac(source_points_3, destination_points_3)
3 merged_image_3 = fun.combined_images(merged_image_2, image4, _H3, focus=2, blend=True)
4 merged_image_3 = fun.remove_black_background((merged_image_3))
5 homography_matrix_list.append(_H3)
6 print(_H3)
7 show_image(merged_image_3, axis=True)
```

```
→ [[ 8.34108545e-01  8.96768938e-02 -5.20390835e+02]
 [-2.24439410e-02  9.26450030e-01 -1.57685494e+02]
 [-1.89723693e-04  2.46754296e-04  1.00000000e+00]]
```



```
1 source_points_4, destination_points_4, matching_result_4 = fun.source_destination_points(merged_image_3, image5)
2 _H4 = fun.ransac(source_points_4, destination_points_4)
3 merged_image_4 = fun.combined_images(merged_image_3, image5, _H4, focus=2, blend=True)
4 merged_image_4 = fun.remove_black_background((merged_image_4))
5 homography_matrix_list.append(_H4)
6 print(_H4)
7 show_image(merged_image_4, axis=True)
```

```
→ [[ 5.07799619e-01 -5.76792684e-02  3.95961583e+01]
 [ 1.37765843e-02  7.40931681e-01 -1.76332390e+02]
 [-4.72498165e-04  7.24826019e-05  1.00000000e+00]]
```



Final I6

```
1 print((homography_matrix_list))
2 show_image(merged_image_4, axis=False)
```

```
[array([[ 1.25680531e+00, -2.33362627e-02, -2.60638267e+02],
        [ 8.03411856e-02,  1.14041527e+00,  1.47499222e+01],
        [ 4.41173699e-04, -1.17344720e-04,  1.00000000e+00]]), array([[ 1.49735477e+00,  2.77822180e-03, -7.44066307e+02],
        [ 9.72082085e-02,  1.39715295e+00, -8.50357001e+01],
        [ 5.49360049e-04,  4.12790562e-05,  1.00000000e+00]]), array([[ 8.34108545e-01,  8.96768938e-02, -5.20390835e+02],
        [-2.24439410e-02,  9.26450030e-01, -1.57685494e+02],
        [-1.89723693e-04,  2.46754296e-04,  1.00000000e+00]]), array([[ 5.07799619e-01, -5.76792684e-02,  3.95961583e+01],
        [ 1.37765843e-02,  7.40931681e-01, -1.76332390e+02],
        [-4.72498165e-04,  7.24826019e-05,  1.00000000e+00]])]
```



End